CSE 3100 | Harder Coding FRQs and Reusable Templates

Original practice exercises grounded in Part 2 Lectures 1-4 and the supplied practice exam; these are not exam predictions.

> **Purpose:** An open-book, typing-friendly coding reference. Five original harder CSE 3100 problems, full commented solutions, expected outputs, common variations, and five reusable starting templates. These are practice problems, not guaranteed exam questions.

> **Scope:** Part 2 Lectures 1-4 (processes, `fork`, `exec`, `open/read`, file-descriptor redirection, and pipes) plus the supplied practice exam.

Fast exam lookup

What the question says	Start here	First thing to draw/check
Parent, Child 1, Child 2, multiple stages and an acknowledgment	**FRQ 1 / Template B**	Draw each directed communication arrow
Parent sends requests; Child replies, then sends text	**FRQ 2 / Template A**	Two pipe directions and message formats
Two shell commands with <, >, 2>, or a command pipeline	**FRQ 3 / Templates C + E**	Which FDs 0, 1, 2 must reference
Many children compute on an array/matrix	**FRQ 4 / Template D**	Each child's assigned work and record ID
Child analyzes a file; Parent writes a report	**FRQ 5 / Templates A + E**	File input, one pipe, stdout redirect

How to use the answers: First try typing the code from the short 'How to solve' steps; then compare your program with the complete version. The full solutions prioritize readable exam patterns. The short integer-transfer helpers check for failures but do not retry short or interrupted transfers unless otherwise noted.

How to study

1. Read the problem statement. 2. Draw processes and arrows. 3. Type the solution once without copying. 4. Compare against the commented answer. 5. Change one variation and recompile.

Pattern lookup

- **Three-process reply:** FRQ 1 / template B
- **Two-way requests:** FRQ 2 / template A

- **exec plus redirection:** FRQ 3 / template C
- **Parallel computation:** FRQ 4 / template D
- **File reads and stdout redirection:** FRQ 5 / template E

FRQ 1: Three processes + four pipes + a feedback acknowledgment

Difficulty: 45-55 points | Very hard

Problem: Create Parent, C1, and C2. Parent sends 2, 5, and 7 one at a time to C1. C1 multiplies by 3 and sends to C2. C2 adds 2 and sends to Parent. Parent replies to C2 with 1 if the final value is even, otherwise 0. C2 counts approvals. **Communication:** P -> C1 -> C2 -> P -> C2 (4 directed pipes)

How to solve

1. Draw four directed arrows and give each a separate pipe.
2. Create all four pipes before either fork.
3. In C1 keep only P->C1 read and C1->C2 write.
4. In C2 keep C1->C2 read, C2->P write, P->C2 read.
5. In Parent keep P->C1 write, C2->P read, P->C2 write.
6. Do each round in the stated order: send, transform, return, acknowledge; close used ends and reap children.

Fully commented solution

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

// A pipe carries bytes. These helpers send/receive one int.
// Short-transfer handling is omitted for this small exam example.
static void send_int(int fd, int x) {
    if (write(fd, &x, sizeof x) != (ssize_t)sizeof x) {
        perror("write");
        exit(1);
    }
}
static int get_int(int fd) {
    int x;
    if (read(fd, &x, sizeof x) != (ssize_t)sizeof x) {
        fprintf(stderr, "Failed to read integer\n");
        exit(1);
    }
    return x;
}
int main(void) {
    // Four directed channels:
    // Parent -> C1 -> C2 -> Parent -> C2 (acknowledgment).
    int p2c1[2], c1c2[2], c2p[2], p2c2[2];
    if (pipe(p2c1) == -1 || pipe(c1c2) == -1 ||
        pipe(c2p) == -1 || pipe(p2c2) == -1) {
        perror("pipe");
        return 1;
    }
    pid_t c1 = fork();
    if (c1 < 0) { perror("fork"); return 1; }
```

```
    if (c1 == 0) {
        // C1 only reads Parent, writes C2.
        close(p2c1[1]); close(c1c2[0]);
        close(c2p[0]); close(c2p[1]);
        close(p2c2[0]); close(p2c2[1]);
        for (int i = 0; i < 3; i++) {
            int x = get_int(p2c1[0]);
            x *= 3;                  // C1 transformation.
            send_int(c1c2[1], x);
        }
        close(p2c1[0]); close(c1c2[1]);
        _exit(0);
    }
    pid_t c2 = fork();
    if (c2 < 0) { perror("fork"); return 1; }
    if (c2 == 0) {
        // C2 reads from C1 and Parent; writes to Parent.
        close(p2c1[0]); close(p2c1[1]);
        close(c1c2[1]); close(c2p[0]); close(p2c2[1]);
        int approved = 0;
        for (int i = 0; i < 3; i++) {
            int x = get_int(c1c2[0]);
            x += 2;                  // C2 transformation.
            send_int(c2p[1], x);
            int ack = get_int(p2c2[0]); // Wait for Parent's decision.
            approved += ack;
        }
        // Use stderr so the parent result lines remain clear.
        fprintf(stderr, "Child 2 approvals: %d\n", approved);
        close(c1c2[0]); close(c2p[1]); close(p2c2[0]);
        _exit(0);
    }
    // Parent writes C1 and C2; reads from C2.
    close(p2c1[0]); close(c1c2[0]); close(c1c2[1]);
    close(c2p[1]); close(p2c2[0]);
    int inputs[3] = {2, 5, 7};
    for (int i = 0; i < 3; i++) {
        send_int(p2c1[1], inputs[i]);
        int value = get_int(c2p[0]);
        int ack = value % 2 == 0;       // 1 for even result.
        printf("Input %d -> result %d -> ACK %d\n", inputs[i], value, ack);
        send_int(p2c2[1], ack);         // Unblocks C2.
    }
    close(p2c1[1]); close(c2p[0]); close(p2c2[1]);
    waitpid(c1, NULL, 0); waitpid(c2, NULL, 0);
    return 0;
}
```

Expected result

```
Input 2 -> result 8 -> ACK 1
Input 5 -> result 17 -> ACK 0
Input 7 -> result 23 -> ACK 0
Child 2 approvals: 1  [printed to stderr; order may vary]
```

Variations

- **Change arithmetic:** Change x *= 3 in C1 and x += 2 in C2.
- **Change rounds:** Change every loop bound and the Parent inputs array.
- **Remove ACK:** Remove p2c2, its descriptor operations and C2 acknowledgment read.
- **Send chars:** Use char and byte-sized transfers instead of int and sizeof int.

Common mistake: C2 must read the acknowledgment only after it has sent the result. Otherwise a blocking cycle can occur.

FRQ 2: Two-way calculator with binary results and a text summary

Difficulty: 35-45 points | Hard

Problem: Parent sends integers 2, 3, 4 over a request pipe. Child squares even inputs and cubes odd inputs, returning each binary int. Parent sends -1 as a sentinel; Child then sends a newline-terminated ASCII summary and exits. **Communication:** P -> Child request pipe; Child -> P result/summary pipe

How to solve

1. Create the two pipes and fork one child.
2. Child closes its request write end and response read end.
3. Child reads until the -1 sentinel, transforms each integer, and sends one binary result.
4. After the sentinel, Child writes an ASCII line terminated by newline.
5. Parent performs send/read for each request, sends -1, then reads characters until newline and waits.

Fully commented solution

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

// Short exam helpers for a single binary int; a production version
// should retry interrupted and partial transfers.
static void send_int(int fd, int x) {
    if (write(fd, &x, sizeof x) != (ssize_t)sizeof x) {
        perror("write"); exit(1);
    }
}
static int get_int(int fd) {
    int x;
    if (read(fd, &x, sizeof x) != (ssize_t)sizeof x) {
        fprintf(stderr, "Read integer failed\n"); exit(1);
    }
    return x;
}
int main(void) {
    int p2c[2], c2p[2];
    if (pipe(p2c) == -1 || pipe(c2p) == -1) {
        perror("pipe"); return 1;
    }
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {
        close(p2c[1]); close(c2p[0]);
        int count = 0;
        while (1) {
            int x = get_int(p2c[0]);
            if (x == -1) break;         // Sentinel: terminate requests.
            int result = x % 2 == 0 ? x * x : x * x * x;
            send_int(c2p[1], result);  // Binary response.
            count++;
        }
        // Send a newline-terminated text summary, not a C '\0'.
```

```
        char msg[80];
        int n = snprintf(msg, sizeof msg, "Processed %d requests.\n", count);
        if (n < 0 || n >= (int)sizeof msg ||
            write(c2p[1], msg, (size_t)n) != n) {
            fprintf(stderr, "Final message failed\n"); _exit(1);
        }
        close(p2c[0]); close(c2p[1]);
        _exit(0);
    }
    close(p2c[0]); close(c2p[1]);
    int values[3] = {2, 3, 4};
    for (int i = 0; i < 3; i++) {
        send_int(p2c[1], values[i]);
        int result = get_int(c2p[0]);
        printf("Input %d -> %d\n", values[i], result);
    }
    send_int(p2c[1], -1);             // Tell child no more requests.
    printf("Child: ");
    char ch;
    while (read(c2p[0], &ch, 1) == 1) {
        putchar(ch);
        if (ch == '\n') break;
    }
    close(p2c[1]); close(c2p[0]);
    waitpid(pid, NULL, 0);
    return 0;
}
```

Expected result

```
Input 2 -> 4
Input 3 -> 27
Input 4 -> 16
Child: Processed 3 requests.
```

Variations

- **Different termination:** Change -1 to any sentinel excluded from valid requests.
- **Different operation:** Replace the even/odd branch with factorial, double, absolute value, etc.
- **Return two integers:** Write two binary values per request and read both in the same order.
- **Use arbitrary text:** Read until newline/EOF and enforce any required maximum length.

Common mistake: The parent must not try to read the final ASCII text as an integer. The protocol switches message formats after the sentinel.

FRQ 3: Two-command pipeline with stdin, stdout, stderr redirection

Difficulty: 45-55 points | Very hard

Problem: Implement the equivalent of grep -i error < input.txt | wc -l > count.txt, but redirect grep error messages to errors.txt. Create exactly two children, use one pipe, and print each child exit status in Parent.
Communication: input file -> grep -> pipe -> wc -l -> count file; grep stderr -> errors file

How to solve

1. Create one pipe before forking.

2. In grep child, open input and errors files; dup2 input to FD 0, pipe write to FD 1, errors to FD 2.
3. In wc child, open count file; dup2 pipe read to FD 0 and file to FD 1.
4. Close original FDs in both children, exec the commands, and exit on exec failure.
5. Parent closes both pipe ends and calls waitpid for both specific children.

Fully commented solution

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

static void die(const char *s) { perror(s); exit(1); }
int main(int argc, char **argv) {
    // ./program input.txt count.txt errors.txt
    if (argc != 4) {
        fprintf(stderr, "Usage: %s INPUT COUNT ERRORS\n", argv[0]);
        return 1;
    }
    int p[2];
    if (pipe(p) == -1) die("pipe");

    pid_t grep_pid = fork();
    if (grep_pid < 0) die("fork");
    if (grep_pid == 0) {
        // grep reads from input file (FD 0), writes to pipe (FD 1).
        int in = open(argv[1], O_RDONLY);
        int err = open(argv[3], O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (in < 0 || err < 0) die("open");
        if (dup2(in, STDIN_FILENO) == -1 ||
            dup2(p[1], STDOUT_FILENO) == -1 ||
            dup2(err, STDERR_FILENO) == -1) die("dup2");
        close(in); close(err);
        close(p[0]); close(p[1]);
        execlp("grep", "grep", "-i", "error", (char *)NULL);
        perror("execlp"); _exit(127);
    }

    pid_t wc_pid = fork();
    if (wc_pid < 0) die("fork");
    if (wc_pid == 0) {
        // wc reads grep's pipe (FD 0), writes to count file (FD 1).
        int out = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (out < 0) die("open");
        if (dup2(p[0], STDIN_FILENO) == -1 ||
            dup2(out, STDOUT_FILENO) == -1) die("dup2");
        close(out);
        close(p[0]); close(p[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("execlp"); _exit(127);
    }
    // Parent must close both ends so wc can observe EOF.
    close(p[0]); close(p[1]);
    int s1, s2;
    if (waitpid(grep_pid, &s1, 0) == -1 ||
        waitpid(wc_pid, &s2, 0) == -1) die("waitpid");
    if (WIFEXITED(s1)) printf("grep status: %d\n", WEXITSTATUS(s1));
    else printf("grep terminated abnormally\n");
    if (WIFEXITED(s2)) printf("wc status: %d\n", WEXITSTATUS(s2));
    else printf("wc terminated abnormally\n");
    return 0;
}
```

Expected result

```
For lines "Error one", "info", "ERROR two", "silent error":
count.txt contains: 3
Parent prints grep status: 0 and wc status: 0
```

Variations

- **Change filter:** Change grep arguments: -n, -v, search word, or explicit file arguments.
- **Change consumer:** Replace wc with sort, uniq, head, etc. and adjust arguments.
- **Append output:** Change O_TRUNC to O_APPEND.
- **Add third stage:** Create a second pipe and another child, like the earlier three-stage pipeline.

Common mistake: Redirection must occur before exec. The parent closes both pipe ends so wc can observe EOF.

FRQ 4: Four workers returning tagged sums and maxima

Difficulty: 40-50 points | Very hard

Problem: A parent has a fixed 4x4 matrix. Create exactly four children, one for each row. Each child calculates its row sum and maximum and sends a struct {row,sum,max} through one shared pipe. Parent prints in row order and calculates the grand total. **Communication:** 4 children -> one shared pipe -> Parent; each record includes its row ID

How to solve

1. Create one pipe before the loop.
2. In each loop iteration fork one child; each child closes read end, computes, sends one tagged struct, then _exit(0).
3. Parent closes the write end after creating all children.
4. Parent receives four complete records; store results at results[r.row].
5. Reap children; print results and total in row order.

Fully commented solution

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

typedef struct {
    int row;    // Tag which child sent the record.
    int sum;    // Sum of row values.
    int max;    // Largest value in row.
} Result;

int main(void) {
    int a[4][4] = {
        {2, 8, 1, 3}, {4, 4, 9, 0},
        {7, 5, 6, 2}, {1, 10, 3, 3}
    };
    int p[2]; pid_t ids[4];
    if (pipe(p) == -1) { perror("pipe"); return 1; }
    for (int i = 0; i < 4; i++) {
```

```
        ids[i] = fork();
        if (ids[i] < 0) { perror("fork"); return 1; }
        if (ids[i] == 0) {
            // Each child writes exactly one short atomic record.
            close(p[0]);
            Result r = {i, 0, a[i][0]};
            for (int j = 0; j < 4; j++) {
                r.sum += a[i][j];
                if (a[i][j] > r.max) r.max = a[i][j];
            }
            if (write(p[1], &r, sizeof r) != (ssize_t)sizeof r) {
                fprintf(stderr, "Write failed\n"); _exit(1);
            }
            close(p[1]); _exit(0);    // Avoid forking more children.
        }
    }
    close(p[1]);
    Result results[4];
    for (int i = 0; i < 4; i++) {
        Result r;
        // Read the exact record length; short reads can happen.
        char *q = (char *)&r;
        size_t left = sizeof r;
        while (left != 0) {
            ssize_t n = read(p[0], q, left);
            if (n <= 0) { fprintf(stderr, "Read failed\n"); return 1; }
            q += n; left -= (size_t)n;
        }
        if (r.row < 0 || r.row >= 4) { fprintf(stderr, "Bad row\n"); return 1; }
        results[r.row] = r;           // Preserve row order.
    }
    close(p[0]);
    for (int i = 0; i < 4; i++) waitpid(ids[i], NULL, 0);
    int total = 0;
    for (int i = 0; i < 4; i++) {
        printf("Row %d: sum=%d max=%d\n", i, results[i].sum, results[i].max);
        total += results[i].sum;
    }
    printf("Grand total: %d\n", total);
    return 0;
}
```

Expected result

```
Row 0: sum=14 max=8
Row 1: sum=17 max=9
Row 2: sum=20 max=7
Row 3: sum=17 max=10
Grand total: 68
```

Variations

- **Find minimum:** Initialize min to first element; compare with <.
- **Compute averages:** Divide sums by 4.0 in the parent.
- **Change number of workers:** Update matrix dimensions, pid array, results array and loop bounds.
- **Different task:** Keep the tagged result structure; change only each child computation.

Common mistake: One shared pipe does not preserve which child finishes first. Never assume the ith received result belongs to row i.

FRQ 5: Child reads a file; Parent redirects a generated report

Difficulty: 35-45 points | Hard

Problem: Child reads a text file with open/read and counts bytes, commas, and logical lines. Child sends a struct with those counts to Parent over one pipe. Parent writes a report by redirecting its own stdout to a supplied report filename with dup2. **Communication:** input file -> Child analysis -> pipe -> Parent -> report file via stdout

How to solve

1. Create pipe and fork.
2. Child closes pipe read end, opens the input file, counts bytes/commas/newlines using read(fd,&c,1).
3. If nonempty input lacks a trailing newline, count the final logical line.
4. Child writes one fixed-size struct and exits.
5. Parent closes write end, reads struct, waits, opens report, dup2(out, STDOUT_FILENO), closes original FD, and prints.

Fully commented solution

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

typedef struct {
    int lines;
    int commas;
    int bytes;
} Metrics;

int main(int argc, char *argv[]) {
    // ./program source.txt report.txt
    if (argc != 3) {
        fprintf(stderr, "Usage: %s SOURCE REPORT\n", argv[0]);
        return 1;
    }
    int p[2];
    if (pipe(p) == -1) { perror("pipe"); return 1; }
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {
        close(p[0]);
        int fd = open(argv[1], O_RDONLY);
        if (fd < 0) { perror("open input"); _exit(1); }
        Metrics m = {0, 0, 0};
        char c, last = '\0';
        ssize_t n;
        while ((n = read(fd, &c, 1)) == 1) {
            m.bytes++;
            if (c == ',') m.commas++;
            if (c == '\n') m.lines++;
            last = c;
        }
        if (n == -1) { perror("read"); _exit(1); }
        // Count a final unterminated line, if present.
        if (m.bytes > 0 && last != '\n') m.lines++;
        close(fd);
        if (write(p[1], &m, sizeof m) != (ssize_t)sizeof m) {
            perror("write"); _exit(1);
```

```
        }
        close(p[1]); _exit(0);
    }
    close(p[1]);
    Metrics m;
    if (read(p[0], &m, sizeof m) != (ssize_t)sizeof m) {
        fprintf(stderr, "Read failed\n"); return 1;
    }
    close(p[0]);
    if (waitpid(pid, NULL, 0) == -1) { perror("waitpid"); return 1; }
    // Redirect only Parent's stdout to report.txt.
    int out = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out == -1) { perror("open report"); return 1; }
    if (dup2(out, STDOUT_FILENO) == -1) { perror("dup2"); return 1; }
    close(out);
    printf("Bytes: %d\nLines: %d\nCommas: %d\n", m.bytes, m.lines, m.commas);
    return 0;
}
```

Expected result

```
For source "one,two\nthree,four" (no final newline):
Bytes: 18
Lines: 2
Commas: 2
```

Variations

- **Count words:** Track transitions between whitespace and non-whitespace.
- **Change delimiter:** Replace c == comma with colon, semicolon, or other character.
- **Write stderr report:** Use dup2(out, STDERR_FILENO) and fprintf(stderr,...).
- **Display on terminal too:** Print before redirecting, or save/restore stdout with dup/dup2.

Common mistake: Only Parent redirects its stdout. Child just reads the file and sends data. Remember logical line handling for a missing trailing newline.

Reusable general templates

Template A: Fixed-size binary request/response

```
// Create these BEFORE fork():
int p2c[2], c2p[2];
pipe(p2c); pipe(c2p);
pid_t pid = fork();
if (pid == 0) {
    close(p2c[1]); close(c2p[0]);
    // Child: read(p2c[0],...), calculate,
    //        write(c2p[1],...) in a loop.
    close(p2c[0]); close(c2p[1]); _exit(0);
}
close(p2c[0]); close(c2p[1]);
// Parent: write(p2c[1],...), read(c2p[0],...)
close(p2c[1]); close(c2p[0]);
waitpid(pid, NULL, 0);
```

Template B: Three-process directed communication

```
int p2c1[2], c1c2[2], c2p[2];
pipe(p2c1); pipe(c1c2); pipe(c2p);
pid_t c1 = fork();
if (c1 == 0) {
    // Keep p2c1[0], c1c2[1]; close others.
    // Read, transform, send; then _exit(0).
}
pid_t c2 = fork();
if (c2 == 0) {
    // Keep c1c2[0], c2p[1]; close others.
    // Read, transform, send; then _exit(0).
}
// Parent keeps p2c1[1], c2p[0]; close others.
// Send input; read final result; wait both.
```

Template C: Command A | Command B with redirection

```
int p[2]; pipe(p);
pid_t a = fork();
if (a == 0) {
    dup2(p[1], STDOUT_FILENO);
    close(p[0]); close(p[1]);
    execlp("commandA", "commandA", (char *)NULL);
    perror("execlp"); _exit(127);
}
pid_t b = fork();
if (b == 0) {
    dup2(p[0], STDIN_FILENO);
    close(p[0]); close(p[1]);
    execlp("commandB", "commandB", (char *)NULL);
    perror("execlp"); _exit(127);
}
close(p[0]); close(p[1]);
waitpid(a, NULL, 0); waitpid(b, NULL, 0);
```

Template D: Multiple workers, tagged records

```
typedef struct { int id; int value; } Result;
int p[2]; pipe(p);
for (int i = 0; i < N; i++) {
    pid_t pid = fork();
    if (pid == 0) {
        close(p[0]);
        Result r = {i, /* compute value for i */ 0};
        write(p[1], &r, sizeof r);
        close(p[1]); _exit(0);
    }
    // Parent stores PIDs for waitpid later.
}
close(p[1]);
// Read N records; use results[r.id] = r.value.
close(p[0]);
// waitpid for all child PIDs.
```

Template E: Redirect one standard descriptor

```
int fd = open("output.txt",
    O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (fd == -1) { perror("open"); exit(1); }
if (dup2(fd, STDOUT_FILENO) == -1) {
```

```
    perror("dup2"); exit(1);
}
close(fd);
// Now stdout writes to the file.
// Put execlp/execvp here for a child command.
```

Exam submission checklist

- [ ] Count child processes.
- [ ] Match read/write endpoints.
- [ ] Close unused ends.
- [ ] Check file flags and dup2 ordering.
- [ ] Check status return values.
- [ ] Confirm outputs and newline / NULL terminators.
- [ ] Reap all children.

Active typing practice (try without looking at the solutions)

Drill 1 - Four-pipe protocol: Without opening FRQ 1, write the four pipe declarations, both child branches, and the parent branch. In each branch, write its `close()` calls before any `read()` or `write()` calls. Then check your endpoints against the communication arrows.

Drill 2 - Two-way request loop: Start with Template A. Change the operation to `x * x + 1`; use `0` as the stop signal. Explain where the Parent sends the sentinel and why the Child must read it before exiting.

Drill 3 - Two-command pipeline: Start with Template C. Replace the command pair with `cat input.txt | wc -w`. Identify which child redirects stdout and which redirects stdin. Ensure the Parent closes both original pipe ends.

Drill 4 - Four workers: Reproduce FRQ 4 but calculate the row minimum and the number of even elements in each row. Keep an identifying `row` field in every result so arrival order does not matter.

Drill 5 - File report: Adapt FRQ 5 to count semicolons and lines. Change the output file from overwrite to append using `O_APPEND` instead of `O_TRUNC`.

Final 60-second correctness check

- [ ] Do I create the required number of processes, and do children exit before any parent-only `fork()`?
- [ ] Are pipes created before forking, with `[0]` used for reads and `[1]` for writes?
- [ ] Have I closed unused pipe ends **inside each process**, and all remaining ends when finished?
- [ ] For commands, does `dup2()` happen before `exec()`? Does failed `exec()` print an error and exit?
- [ ] Do I know which messages are binary integers and which are newline-terminated text?
- [ ] If results arrive out of order, does each carry its assigned row/task ID?
- [ ] Are file modes (`O_TRUNC`, `O_APPEND`, `O_CREAT`) consistent with the question?
- [ ] Do reads handle EOF or failure, and does the Parent `waitpid()` for every child?

Note: The actual exam's rules govern whether locally saved prewritten code or reference materials are allowed. A PDF of worked examples does not replace practicing how to modify pipe directions, descriptor cleanup, or execution order.